

1. Menciona 3 beneficios del Diseño basado en dominio
2. Menciona 4 principios clave del modelo Cliente-Servidor
3. Desarrollar donde se aplicarían los criterios de uso del modelo orientado a servicio (SOA)

Un estilo de arquitectura o patrón de arquitectura es un patrón que provee un marco abstracto para una familia de sistemas. Promueve la partición y la reutilización de diseño para dar solución a un conjunto de problemas determinado.

Garlan y Shaw lo definen como un conjunto de componentes conectores y restricciones que pueden ser utilizados dentro de ese estilo.

Entender los diferentes estilos de arquitectura dan como beneficio poder tener un lenguaje común sin entrar en soluciones demasiado específicas.

Según la categoría del problema existen arquitecturas que responden para dar una solución:

categoría	estilo de arquitectura
communication	Services oriented architecture, message bus
Deployment	Client/server, N-tier, 3-Tier
Dominio	Domain driven design
Structure	Component-based, Object oriented, Layered Architecture

Los estilos de arquitectura no son únicos por proyecto, generalmente un sistema aplica una combinación de dos o más estilos. Al descomponer el problema en varias partes, para cada parte podemos aplicar distintos estilos y a su vez uno para que unifique todo el sistema.

Client/Server Architectural Style (Cliente servidor)

1. Principios clave del modelo

La arquitectura cliente servidor está formada por un servidor, un cliente separado y una conexión entre ambos. En su forma más sencilla un servidor es accedido por uno o más clientes, conocido también como un 2-Tier. En este esquema los clientes poseen una interfaz de usuario a través de la cual se comunican con el servidor para realizar peticiones. Por su parte el servidor gestiona las peticiones a través de sus reglas de negocio para acceder y operar la información que aloja.

Algunas variaciones del cliente servidor pueden ser:

Client Queue- Client systems : En este modelo los clientes se comunican entre sí a través del servidor y lo utilizan como intermediario para compartir y alojar información

Peer to peer applications: Este modelo basado en el cliente queue client intercambia el rol cliente servidor de modo que todos los nodos que participan de la conexión puedan sincronizar archivos e información entre dos o más terminales. Esto permite que se multipliquen las

respuestas a las consultas, se comparta información y genera resiliencia para el sistema si se remueven terminales de la comunicación por algún motivo.

Application Servers: Es una arquitectura especializada donde los clientes funcionan como una terminal a través de un navegador o un software especializado para ejecutar operaciones directamente en el servidor.

2. Beneficios

- Seguridad: La información es alojada en el servidor que siempre suele ofrecer mejor seguridad que los clientes.
- Acceso centralizado a la información: Al encontrarse la información en un mismo lugar la misma se actualiza con mucha más facilidad que en otros tipos de arquitecturas.
- Fácil de mantener: Los roles y responsabilidades de un sistema están distribuidos en varios servidores que son conocidos entre sí en la red, esto asegura a los clientes que no se vean afectados por la reparación, actualización o relocalización de un servidor.

3. Aplicaciones

El modelo cliente servidor debe ser considerado en los los siguientes casos

- Basado en un servidor que deba responder a muchos clientes
- Aplicación web que será expuesta a través de un navegador
- Implementar procesos de negocio que serán utilizados por toda la organización
- Crear servicios que sean consumidos por otras aplicaciones.
- Centralizar la información, los backup o la administración de funciones.
- La aplicación debe soportar clientes de diferente tipo o diferentes dispositivos

Component-Based Architectural Style (Basado en componentes)

La arquitectura basada en componentes desglosa el diseño en componentes con una funcionalidad o lógica bien definida a través de la comunicación de las interfaces que contiene métodos, eventos y propiedades. Ofrece un nivel de abstracción mayor que el orientado a objetos y no hace focos en problemas como los protocolos de comunicación y un estado compartido.

1. Principios clave del modelo

El principio de este estilo se basa en sus componentes que tienen las siguientes características:

- Reusables: Los componentes se diseñan para ser reutilizados en diferentes escenarios y aplicaciones, pero existen casos de componentes específicos para una tarea particular.
- Reemplazables: Son fácilmente reemplazables por otros componentes
- Independencia del contexto: Los componentes son diseñados para funcionar en diferentes ambientes y contextos. La información necesaria le debe ser provista al componente en lugar de incluirla en él o pedirle que la acceda.
- Extensible: Un componente puede ser extendido desde componentes existentes y agregarle comportamiento
- Encapsulamiento: Los componentes exponen interfaces que permiten que otros utilicen su funcionalidad, protegiendo su estado, comportamiento y variables internas.

- Independencia: los Componentes son diseñados para tener las mínimas dependencias posibles en otros componentes. De esta forma los componentes se pueden desplegar en otros sistemas sin comprometer al sistema en sí o algún componente del mismo.

CLASIFICACIÓN DE COMPONENTES PÁGINA 24

2. Beneficios

- Fácil de desplegar: Así como se desarrollen nuevos componentes compatibles se pueden reemplazar los existentes sin impactar en otros o en el sistema en sí
- Reducción de costos: La posibilidad de utilizar componentes de terceros permiten ahorrar el costo de desarrollo y mantenimiento
- Fácil de desarrollar: Los componentes implementados con interfaces definidas proveen una funcionalidad definida, permitiendo desarrollar sin impactar en otras partes del sistema.
- Mitigar complejidad técnica: Los componentes mitigan la complejidad de usar un componente contenedor y sus servicios.

3. Aplicaciones

Los patrones de diseño como Dependency Injection of Service Locator manejan la dependencia entre componentes promoviendo el bajo acoplamiento y la reutilización. Estos patrones son usados para construir aplicaciones compuestas que combinan y usan componentes para múltiples aplicaciones.

Este estilo de arquitectura debe ser considerado en caso de:

- Si ya se poseen componentes o es posible utilizar componentes de terceros.
- Utilizar componentes diseñados en lenguajes diferentes
- Tener una aplicación con una arquitectura compuesta que le sea fácil reemplazar o actualizar componentes individuales.

Domain Driven Design Architectural Style

1. Principios clave del modelo

Es un modelo de enfoque similar a orientación a objetos, pero en este caso la aproximación es hacia el dominio del negocio particular. Los elementos, comportamientos y relaciones serán una imagen de la empresa a la cual el software pretende dar solución. Para aplicar este modelo se debe poseer un conocimiento del dominio de negocio que se quiere modelar. El equipo de desarrollo debe trabajar codo a codo con los expertos del modelo de negocio de manera, manejar un lenguaje común excluyendo la jerga técnica.

Lograr la tarea de que el equipo de desarrollo y los expertos del área de su aplicación manejen el mismo lenguaje este modelo debe ser utilizado en los escenarios donde exista una gran complejidad en el dominio del negocio.

2. Beneficios

- Comunicación: Todas las partes manejan un lenguaje único para comunicarse de modo que la misma se realiza sin utilizar una jerga técnica

- Extensible: Este modelo suele ser modular y flexible lo que permite su actualización y extensión con facilidad.
- Testable: Este modelo contiene bajo acoplamiento y cohesión lo que facilita su testeo

3. Aplicaciones

- Dominos complejos donde se desee una comunicación fluida con el equipo de desarrollo, o se desee que la aplicación se desarrolle bajo un lenguaje que todos los interesados comprendan
- Si existe una larga y compleja organización que se dificulte de manejar con otros tipos de modelos.

Layered Architecture

La arquitectura en capas consta de agrupar funcionalidad en capas de manera vertical. La agrupación se da por un role común o responsabilidad, la comunicación entre capas es explícita y con bajo acoplamiento. Esta forma de agrupación proporciona flexibilidad y mantenibilidad. Este modelo puede ser descrito como una pirámide invertida donde cada capa agrega funcionalidad a la capa inferior. Cada componente puede comunicarse solo con componentes de su misma capa o de la capa inferior. Algunos modelos menos estrictos permiten que el componente de una capa se conecte con capas más inferiores a la inmediata. Las capas pueden estar en una misma computadora o pueden estar distribuidas en diferentes computadoras, los componentes entre capas se comunican a través de interfaces bien definidas. Un ejemplo de capas podría ser una aplicación web que contiene una capa de interfaz de usuario, una capa de lógica de negocio y una capa de datos.

1. Principios clave del modelo

- Abstracción: El modelo de capas permite ver al sistema como un todo a la vez que proporciona un detalle para entender los roles y responsabilidades de cada capa y su relación.
- Encapsulamiento: Durante el diseño no es necesario asumir sobre los tipo de datos, métodos propiedades o implementaciones, ya que estas características no lo afectan
- Funcionamiento de capas bien definido: La separación de funcionalidad en cada capa es clara. Las capas superiores como de presentación envían comandos a las inferiores permitiendo a la información ir de arriba hacia abajo.
- Alta cohesión: Estando claramente definida la responsabilidad de cada capa y asegurando que cada una tiene su funcionalidad relacionada a su responsabilidad, ayudará a maximizar la cohesión entre las capas.
- Reusable: las capas inferiores no tienen dependencias con las capas superiores lo que les permite ser reutilizables en otros escenarios.
- Bajo acoplamiento: La comunicación entre capas se basa en la abstracción y eventos provocando bajo acoplamiento entre ellas.

Separated Patterns es un patrón de diseño que utiliza el modelo de capas:

Principios del modelo Separated Patterns:

- Separación de preocupaciones: divide el procesamiento de la interfaz de usuario por ejemplo en el modelo MVC.

- Notificaciones basadas en eventos: The observer pattern es comúnmente usado para brindar notificaciones al usuario a través de la vista cuando la información manejada por el modelo cambia.
- Manejo delegado de eventos: El controlador maneja eventos que son disparados por los controles del usuario desde la vista

Otros ejemplos del Separated Presentation son el Passive View y el Supervising Presenter.

2. Beneficios

- Abstracción: El nivel de abstracción puede ser modificado en el modelo de capas
- Aislamiento: El aislamiento permite actualizar la tecnología en una capa impidiendo el impacto en las demás capas
- Manejabilidad: Separar las preocupaciones ayuda a identificar las dependencias organizando el código de mejor manera
- Performance: La posibilidad de distribuir las capas en diferentes nodos puede mejorar la escalabilidad, la tolerancia al fallo y el rendimiento
- Reusabilidad: Los roles permiten la reusabilidad, en un modelo MVC los controladores pueden ser fácilmente reutilizados para vistas similares o compatibles, brindando la misma información pero con una visualización personalizada.
- Testabilidad: Teniendo bien definida las capas se incrementa la posibilidad de testa. Separated Presentation permite utilizar mocs para imitar el comportamiento de objetos concretos como la vista, el modelo o el controlador durante el testeo.

3. Aplicaciones

- Se poseen capas para reutilizar en otras aplicaciones
- Se tienen aplicaciones que exponen procesos de negocio a través de interfases
- La complejidad exige separar el problema en capas para que el equipo pueda hacer foco en cada una de ellas
- Si la aplicación debe soportar diferentes tipos de clientes y dispositivos

Considerar Separated Presentation:

- Si se quiere utilizar testeos ágiles y simplificar el mantenimiento de la interfaz de usuario.
- Si se quiere separar la interfaz de usuario de la lógica de negocio para desarrollarlas por separado.

Message Bus

Este modelo se basa en la capacidad de un sistema para enviar y recibir mensajes a través de uno o más canales de comunicación, de esta manera las aplicaciones pueden comunicarse sin necesidad de saber mucho una sobre otra. Las aplicaciones envían mensajes asincrónicos a través de un bus de uso común. La mayoría de las implementaciones utiliza un enrutador de mensajes o un patrón de publicación y suscripción y a la vez implementan un sistema de mensajería como Message Queuing. La mayoría de las implementaciones consisten en aplicaciones individuales que utilizan esquemas similares y comparten infraestructura para enviar y recibir mensajes.

1. Principios clave del modelo

Este modelo provee la habilidad de manejar:

- Mensajes orientados a la comunicación: Toda comunicación entre aplicaciones está basada en mensajes que usan un esquema conocido
- Lógica de procesamiento complejo: Las operaciones complejas se pueden ejecutar combinando pequeñas operaciones
- Modificación de la lógica de proceso: como la interacción con el bus está basada en esquemas y comandos comunes, se pueden insertar o remover aplicaciones al bus para modificar la lógica que se usa para procesar los mensajes
- Integración con diferentes ambientes: Usando una comunicación basada en mensajes se puede interactuar con aplicaciones desarrolladas en diferentes ambientes tales como java y .NET

Algunas variaciones a este modelo son

Enterprise Service Bus este modelo permite la comunicación entre un componente y el bus de mensajes permitiendo a aplicaciones que utilizan otro tipo de formato de mensajes poder sumarse al bus de mensajes.

Internet Service Bus: Es similar al enterprise service bus pero permite utilizar aplicaciones que están alojadas en la nube en lugar de una lan. Este modelo utiliza Uniform Resource Identifiers(URIs) y políticas para controlar el enrutamiento lógico a través de las aplicaciones y servicios en la nube.

2. Beneficios

- Extensibilidad: Aplicaciones pueden ser agregadas o removidas del bus sin impactar en el resto.
- Baja complejidad: La complejidad es baja, ya que la aplicación solo debe comunicarse con el bus
- Flexibilidad: Las aplicaciones que realizan complejos procesos o las comunicaciones entre aplicaciones pueden ser fácilmente modificadas cambiando la configuración de enrutamiento.
- Bajo acoplamiento: Mientras que la aplicación tenga una comunicación clara con el bus no existe dependencia alguna permitiendo cambios, actualizaciones e incluso reemplazarla siempre que mantenga la misma interfase
- Escalabilidad: Múltiples instancias de la misma aplicación pueden ser agregadas al bus para manejar múltiples consultas al mismo tiempo.
- Simplicidad de aplicación: Aunque el bus de mensajes agrega complejidad a la infraestructura, cada aplicación debe ocuparse únicamente de una sola comunicación al bus en lugar de tener múltiples comunicaciones.

3. Aplicaciones

- Si existen múltiples aplicaciones que interoperan entre ellas para realizar tareas
- Si se desean cambiar múltiples tareas en una unica operacion
- Implementar una tarea que requiere interacción con aplicaciones externas o aplicaciones alojadas en diferentes ambientes

N-Tier /3-Tier

1. Principios clave del modelo

Es un modelo muy similar al de capas, pero cada capa se aloja en una computadora física diferente. Utilizan una plataforma específica como método de comunicación en lugar de un enfoque basado en mensajes.

Este modelo está caracterizado por la descomposición de aplicaciones, servicios de componentes y un desarrollo distribuido, de esta manera provee mayor escalabilidad, disponibilidad y mantenibilidad a la vez que una mejor utilización de los recursos. Cada nodo es independiente del resto excepto de aquel que está por encima y por debajo de él. El nodo n solo debe saber cómo manejar peticiones del nodo $n+1$, reenviar dicha petición al nodo $n-1$ si es que existe y cómo manejar los resultados de esa petición. La comunicación entre nodos es usualmente asíncrona para proporcionar mejor escalabilidad.

Usualmente tiene tres partes lógicas separadas, cada una separada en un servidor físico diferente. Cada parte es responsable de una funcionalidad específica.

2. Beneficios

- **Mantenibilidad:** Como cada nodo es independiente del resto, las actualizaciones y cambios pueden ser llevadas a cabo sin afectar al sistema.
- **Escalabilidad:** Como los nodos son basados en el despliegue de capas, la escalabilidad es posible.
- **Flexibilidad:** Como cada nodo puede ser manejado o escalado independientemente del resto, la flexibilidad aumenta.
- **Disponibilidad:** Las aplicaciones pueden explotar la arquitectura modular estableciendo sistemas con componentes fácilmente escalables, aumentando la disponibilidad.

3. Aplicaciones

- Considerar el modelo n-tier o 3-tier si el procesamiento de una capa puede afectar el rendimiento de otras capas alojadas en el mismo servidor.
- La capa de negocio debe ser montada detrás de un firewall por seguridad, lo que obliga a montar la capa de presentación en una máquina diferente.
- Si se desea compartir la lógica de negocio entre dos aplicaciones y los recursos de hardware lo permiten.
- Usar 3-tier si se está desarrollando una aplicación donde los servidores están alojados en una red privada empresarial o en una aplicación de internet donde los requerimientos de seguridad no restringen el despliegue de lógica de negocio en la aplicación del servidor.
- Considerar usar más de 3-tier si los requerimientos de seguridad dictaminan que la lógica de negocio no puede estar en el perímetro de la red o la aplicación tiene una exigencia de recursos que precise descargar funcionalidad en otro servidor.

Object Oriented

Este tipo de modelo está basado en la división de responsabilidad en objetos individuales, reusables y autosuficientes, que contienen información y comportamiento que sólo es relevante

para el objeto. Este modelo puede ser visto como un conjunto de objetos que cooperan entre sí en vez de un set de rutinas de instrucciones. Los objetos son discretos, independientes con baja aplicación y se comunican a través de interfaces a través de métodos que le permiten acceder a las propiedades de dichos objetos enviando y recibiendo mensajes.

1. Principios clave del modelo

- **Abstracción:** Permite reducir una operación compleja a una generalización que contiene la base de dicha operación, por ejemplo una interfaz abstracta bien definida permite acceso a la información a través de métodos get y update.
- **Composición:** Los objetos pueden ser ensamblados por otros objetos y pueden elegir ocultar esos objetos a otras clases o exponerlos a través de interfaces
- **Herencia:** Los objetos pueden heredar atributos y comportamiento de otros objetos e implementar nuevos comportamientos. Esto facilita el mantenimiento, ya que modifican el objeto base, los objetos hijos se actualizan inmediatamente.
- **Encapsulamiento** Los objetos exponen funcionalidad solo a través de métodos, propiedades y eventos ocultando los detalles internos de otros objetos. Mientras las interfaces se mantengan compatibles esto facilita modificar el comportamiento interno.
- **Polimorfismo** Esto permite sobrescribir el comportamiento base implementando nuevos tiempos sobre un objeto existente
- Los objetos pueden ser desacoplados por el consumidor definiendo interfaces abstractas que el objeto implementa y el consumidor comprende. Esto permite modificar las implementaciones sin afectar al consumidor de la interfase

El uso más común de este modelo es para problemas donde resulte útil modelar objetos del mundo real

2. Beneficios

- **Comprensible** Mapear la realidad con la aplicación hace que el sistema sea fácilmente entendible
- **Reusable** provee usabilidad a través del polimorfismo y la abstracción
- **Testable** provee facilidad en el testeo gracias al encapsulamiento
- **Extensible:** la encapsulación y el polimorfismo y la abstracción asegura que los cambios en la representación de datos no afecte las interfaz que los objetos exponen.
- **Altamente cohesivo.** Localizando los métodos en un objeto y usando diferentes objetos para diferentes cosas se obtiene un alto nivel de cohesión

3. Aplicaciones

- Considerar la orientación a objetos si la aplicación está basada en un problema del mundo real.
- Si ya se tienen objetos y clases que coinciden con el diseño requerido
- Si es necesario encapsular lógica e información en componentes reutilizable
- Si la lógica de negocio es complicada y se requiere un buen nivel de abstracción.

Service Oriented Architecture

Este modelo permite que la funcionalidad de las aplicaciones sea provista como un set de servicios. Los servicios están levemente acoplados porque usan estándares basados en interfase que pueden ser invocadas, publicadas o descubiertas. NO ENTENDI

Este estilo puede contener paquetes de procesos de negocios en servicios usando un rango de protocolos y formato de datos para comunicar información. Clientes y otros servicios pueden acceder a servicios locales corriendo en la misma maquina o a traves de acceso remoto

1. Principios clave del modelo

- Los servicios son autónomos: cada servicio es mantenido desarrollado desplegado y versionado independientemente
- Servicios son distribuibles: los servicios pueden ser alojados en cualquier lugar de la red local o remota siempre que la red soporte los requerimientos de comunicación
- Servicios poco acoplados: cada servicio es independiente del resto y pueden ser reemplazados y actualizados sin afectar las aplicaciones que lo utilizan siempre que no cambie la interfase.
- Los servicios comparten esquema y contrato no clase: los servicios comparten contratos y esquemas cuando se comunican, no clases internas
- Compatibilidad basada en políticas: En este contexto las políticas son las características de transporte protocolo y seguridad

Los ejemplos básicos de este tipo de sistemas son los sistemas de reservas o tiendas online

2. Beneficios

- Alineación de dominio: La reutilización de servicios comunes con interfaces estandarizadas incrementa las oportunidades de negocio y reduce los costos
- Abstracción Los servicios son autónomos y se acceden por un contrato formal lo que provee poco acoplamiento y abstracción
- Discoverability: Los servicios pueden exponer descripciones que le permiten a otros servicios y aplicaciones localizarlo y determinar la interfase
- Interoperabilidad: Como los protocolos y el formato de los datos estaba basado en estándares, el proveedor y consumidor del servicio puede ser construido y desplegado en diferentes plataformas
- Racionalización: los servicios pueden ser granulares para proveer funcionalidad específica en lugar de duplicar la funcionalidad en número de aplicaciones lo que remueve la duplicación.

3. Aplicaciones

- Si se tiene acceso a servicios que se quieren reutilizar
- So se pueden utilizar servicios provistos por una compañía
- Se quieren construir aplicaciones que precisan variedad de servicios detrás de una única interfaz de usuario
- Si se está creando Software plus services software as a service o aplicaciones basadas en la nube
- El estilo SOA es adecuado cuando debe admitir la comunicación basada en mensajes entre segmentos de la aplicación y exponer la funcionalidad de una manera independiente de la plataforma,
- Si desea exponer servicios que se pueden descubrir a través de directorios y que pueden ser utilizados por clientes que no tienen conocimiento previo de las interfaces.
- cuando desea aprovechar los servicios federados como la autenticación,